```
            final Person other = (Person) obj;
            if (this.sex != other.sex) {
                return false;
            }
            if (!this.fName.equals(other.fName)) {
                return false;
            }
            return this.sName.equals(other.sName);
        }

        @Override
        public String toString() {
            return "Person{" + "fName=" + fName + ", sName=" + sName + ", sex="
    + (sex==1?"M":"F") + '}';
        }
    }
```

Κώδικας 11.11 *Η κλάση Person*

Στον κώδικα 11.11 παρατίθεται η κλάση Person. Πρόκειται για απλή εφαρμογή της ενσωμάτωσης. Παρόμοιο αποτέλεσμα μπορεί να παραχθεί αυτόματα από το NetBeans ως εξής: Δημιουργήστε μια κλάση Person και προσθέστε τις μεταβλητές στιγμιότυπου. Μεταβείτε στον editor του NetBeans στην κλάση. Με δεξί κλικ ανοίγει ένα μενού. Επιλέξτε Insert Code. Από το υπομενού που θα ανοίξει επιλέξτε σταδιακά την αυτόματη δημιουργία δημιουργών, αναγνωστών, ρυθμιστών, μεθόδου equals, μεθόδου hashcode και μεθόδου toString. Εντοπίστε και μελετήστε τις διαφορές μεταξύ του κώδικα που παράγει το NetBeans και του κώδικα 11.11.

11.9.2 Η κλάση MyInteger

Υλοποιήστε την κλάση MyInteger που περιλαμβάνει τις συναρτήσεις που περιγράφονται παρακάτω και όλες τις αναγκαίες συναρτήσεις.

```
public class MyInteger {
     private int value;
     public MyInteger();
     public MyInteger(int k);
     public MyInteger add(MyInteger i);
     public MyInteger subtract(MyInteger i);
     public MyInteger multiply(MyInteger i);
     public MyInteger divide(MyInteger i);
     public String toString();
     public boolean equals(Object obj);
     public int hashCode();
};
```

Να υπερφορτωθούν οι μέθοδοι setValue, add, subtract, multiply, divide ώστε να δέχονται παραμέτρους τύπου Integer και τύπου int.

Λύση

```
public class MyInteger {

    private int value;

    public MyInteger() {
        value = 0;
    }

    public MyInteger(int k) {
        value = k;
```

```
    }

    public void setValue(int k) {
        value = k;
    }

    public void setValue(Integer k) {
        value = k;
    }

    public void setValue(MyInteger k) {
        value = k.value;
    }

    public int getValue() {
        return value;
    }

    public MyInteger add(MyInteger i) {
        return new MyInteger(value + i.value);
    }

    public MyInteger subtract(MyInteger i) {
        return new MyInteger(value - i.value);
    }

    public MyInteger multiply(MyInteger i) {
        return new MyInteger(value * i.value);
    }

    public MyInteger divide(MyInteger i) {
        return new MyInteger(value / i.value);
    }

    @Override
    public String toString() {
        return Integer.toString(value);
    }

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 61 * hash + this.value;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final MyInteger other = (MyInteger) obj;
        return this.value == other.value;
    }
}
```

Κώδικας 11.12 *Η κλάση MyInteger*

Η κλάση MyInteger (κώδικας 11.12) εκτός από τις αναγκαίες συναρτήσεις περιλαμβάνει και μια σειρά από συναρτήσεις που υποστηρίζουν τις βασικές πράξεις. Η υλοποίησή τους είναι πολύ απλή. Σε σχέση με την απαίτηση της εκφώνησης να υπερφορτωθούν οι συναρτήσεις setValue, add, subtract, multiply, divide, έχουμε υλοποιήσει την υπερφόρτωση μόνο για την setValue. Παρόμοια μπορείτε να υλοποιήσετε και τις υπόλοιπες αναγκαίες μεθόδους.

11.9.3 Μονάδα Μέτρησης Βάρους

Να υλοποιηθεί η κλάση WeightMeasurementUnit (Μονάδα Μέτρησης Βάρους). Η κλάση διαθέτει δύο μεταβλητές στιγμιότυπου, την description τύπου String που αναπαριστά το λεκτικό της μονάδας και την relationToGr τύπου double που αναπαριστά τη σχέση της μονάδας με το γραμμάριο. Να προστεθεί η συνάρτηση main στην οποία να δημιουργηθούν τρία αντικείμενα, ένα που αναπαριστά το χιλιόγραμμο, ένα για τον τόνο και ένα για το χιλιοστό του γραμμαρίου.

Λύση

```
public class WeightMeasurementUnit {

    private String description;
    private double relationToGr;

    public WeightMeasurementUnit(String lekt, double relationToGr) {
        if (relationToGr <= 0) {
            throw new RuntimeException();
        }
        this.description = lekt;
        this.relationToGr = relationToGr;
    }

    public String getDescription() {
        return description;
    }

    public double getRelationToGr() {
        return relationToGr;
    }

    public void setDescription(String description) {
        this.description = description;
    }


    @Override
    public int hashCode() {
        return Double.hashCode(this.relationToGr);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
```

```
        }
        final WeightMeasurementUnit other = (WeightMeasurementUnit) obj;
        if (Double.compare(this.relationToGr, other.relationToGr) != 0) {
            return false;
        }
        return true;
    }

    @Override
    public String toString() {
        return "WeightMeasurementUnit{" + "description=" + description + ",
relationToGr=" + relationToGr + '}';
    }

    public static void main(String[] args) {
        WeightMeasurementUnit kgr = new WeightMeasurementUnit("Kgr", 1000),
                metricTon = new WeightMeasurementUnit("Metric Ton", 1000000),
                mgr = new WeightMeasurementUnit("mgr", 1 / 1000d);
        System.out.println(kgr);
        System.out.println(metricTon);
        System.out.println(mgr);
    }
}
```

Κώδικας 11.13 *Κλάση αναπαράστασης της Μονάδας Μέτρησης Βάρους*

Ο κώδικας 11.13 δίνει τον ορισμό της κλάσης WeightMeasurementUnit. Κάθε τέτοια μονάδα προσδιορίζεται από τη σχέση της με το γραμμάριο. Έτσι στον δημιουργό της κλάσης γίνεται έλεγχος ώστε η σχέση της μονάδας με το γραμμάριο να είναι έγκυρη. Στη συνέχεια λαμβάνουν τιμές τα πεδία της κλάσης και δημιουργείται το αντικείμενο. Η ισότητα βασίζεται αποκλειστικά στη μεταβλητή relationToGr, δηλαδή δύο μονάδες μέτρησης βάρους με τη ίδια σχέση με το γραμμάριο θεωρούνται ίσες ανεξάρτητα από τα υπόλοιπα χαρακτηριστικά τους. Αντίστοιχα και η hashCode βασίζεται αποκλειστικά στηνσχέση με το γραμμάριο.

11.9.4 Βάρος

Να υλοποιηθεί η κλάση Weight. Κάθε αντικείμενο της κλάσης διαθέτει δύο μεταβλητές. Η μια εκφράζει το μέτρο του βάρους και είναι double και η άλλη τη μονάδα μέτρησης του βάρους και είναι WeightMeasurementUnit.
Η κλάση διαθέτει τρεις δημιουργούς ως εξής:

```
public Weight(double value, WeightMeasurementUnit mB)
```

Δημιουργεί αντικείμενο με μέτρο value και μονάδα μέτρησης mB.

```
public Weight(Weight w)
```

Δημιουργεί αντίγραφο του w.

```
public Weight(double value, String unitDescription, double
relationToGr)
```

Δημιουργεί αντικείμενο με μέτρο value και μονάδα μέτρησης με περιγραφή unitDescription και σχέση με το γραμμάριο relationToGr.

Πέραν των αναγκαίων συναρτήσεων, η κλάση διαθέτει και τις ακόλουθες συναρτήσεις:

```
public static Weight convert(WeightMeasurementUnit from,
WeightMeasurementUnit to)
```

Μετατρέπει τη μονάδα μέτρησης from μονάδα μέτρησης to. Προσέξτε πως επιστρέφει Weight. Πράγματι, αν για παράδειγμα μετατρέψω το κιλό σε γραμμάρια θα λάβω ως αποτέλεσμα 1000 γραμμάρια, μέγεθος που

υποχρεωτικά εκφράζεται σε βάρος. Η συνάρτηση είναι στατική καθώς δεν είναι απαραίτητο να διαθέτουμε αντικείμενο Weight για να μετατρέψουμε μια μονάδα σε μία άλλη.

```
public Weight convert(WeightMeasurementUnit wUnit)
```

Μετατρέπει αυτό το βάρος στην μονάδα wUnit.

```
public Weight add(Weight w)
```

Προσθέτει αυτό το βάρος, δηλαδή το βάρος που την καλεί, με το βάρος w. Επιστρέφει το άθροισμα εκφρασμένο στη μονάδα βάρους που καλεί την add.

Λάβετε υπόψη πως η μέθοδος equals θα πρέπει να επιστρέφει true για ίσα βάρη. Για παράδειγμα, το 1 κιλό είναι ίσο με τα 1000 γραμμάρια. Αντίστοιχα, και η μέθοδος hashCode πρέπει να επιστρέφει την ίδια τιμή για βάρη που είναι ίσα.

Προσθέστε στην κλάση, αυτήν την main

```
public static void main(String[] args) {
     WeightMeasurementUnit          kgr          =          new
     WeightMeasurementUnit("kgr", 1000);
     WeightMeasurementUnit          mgr          =          new
     WeightMeasurementUnit("mgr", 1 / 1000d);
     Weight w1 = new Weight(1, kgr);
     Weight w2 = new Weight(100, mgr);
     Weight w3 = w1.add(w2);
     Weight w4 = w2.add(w1);
     System.out.println(w3);
     System.out.println(w4);
     System.out.println(w3.equals(w4));
     System.out.println(w3.hashCode() == w4.hashCode());
}
```

Ελέγξτε ώστε η έξοδος να είναι σύμφωνη με

```
1.0001 kgr
1000100.0 mgr
true
true
```

Λύση

```
public class Weight {

    private double value;
    private WeightMeasurementUnit weightUnit;

    public Weight(double value, String unitDescription,
                  double relationToGr) {
        this(value, new WeightMeasurementUnit(unitDescription,
                  relationToGr));
    }

    public Weight(double value, WeightMeasurementUnit mB) {
        this.value = value;
        this.weightUnit = mB;
    }

    public Weight(Weight w) {
        this(w.value, w.weightUnit);
    }
```

```
public static Weight convert(WeightMeasurementUnit from,
        WeightMeasurementUnit to) {
    double valueTo = from.getRelationToGr() / to.getRelationToGr();
    return new Weight(valueTo, to);
}

public Weight convert(WeightMeasurementUnit wUnit) {
    Weight rVal = convert(weightUnit, wUnit);
    rVal.value *= value;
    return rVal;
}

public Weight add(Weight w) {
    //Weight w1 = new Weight(w);
    if (!weightUnit.equals(w.weightUnit)) {
        w = w.convert(weightUnit);
    }
    return new Weight(value + w.value, weightUnit);
}

public void setValue(double value) {
    this.value = value;
}

public double getValue() {
    return value;
}

public WeightMeasurementUnit getWeightUnit() {
    return weightUnit;
}

public void setWeightUnit(WeightMeasurementUnit weightUnit) {
    this.weightUnit = weightUnit;
}

@Override
public int hashCode() {
    Weight w = convert(new WeightMeasurementUnit("gr", 1));
    int hash = 7;
    hash = 23 * hash + Double.hashCode(w.value);
    return hash;
}

@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    Weight other = (Weight) obj;
    other = other.convert(this.weightUnit);
    return Double.compare(this.value, other.value) == 0;
}
```

```
    @Override
    public String toString() {
        return value + " " + weightUnit.getDescription();
    }

    public static void main(String[] args) {
        WeightMeasurementUnit kgr = new WeightMeasurementUnit("kgr", 1000);
        WeightMeasurementUnit mgr = new WeightMeasurementUnit("mgr", 1 / 1000d);
        Weight w1 = new Weight(1, kgr);
        Weight w2 = new Weight(100, mgr);
        Weight w3 = w1.add(w2);
        Weight w4 = w2.add(w1);
        System.out.println(w3);
        System.out.println(w4);
        System.out.println(w3.equals(w4));
        System.out.println(w3.hashCode() == w4.hashCode());
    }
}
```

Κώδικας 11.14 *Η κλάση Weight*

Για να μετατρέψω μια μονάδα βάρους σε μια άλλη θα πρέπει να αξιοποιήσω την σχέση κάθε μονάδας με το γραμμάριο. Έστω ότι θέλω να μετατρέψω τη μονάδα βάρους from στην to. Η from σε γραμμάρια είναι ένα βάρος με τιμή from.relationToGr και μονάδα το γραμμάριο. Αντίστοιχα, η to σε γραμμάρια είναι ένα βάρος με μέτρο to.relationToGr και μονάδα το γραμμάριο. Τώρα και οι δύο μονάδες είναι εκφρασμένες σε βάρη με μονάδα το γραμμάριο. Επομένως, η διαίρεση from.relationToGr/to.relationToGr μας δίνει το μέτρο του βάρους που εκφράζει τη μονάδα from στην μονάδα to. Αυτή είναι η λογική της convert που μετατρέπει από μια μονάδα σε άλλη, δηλαδή της convert(WeightMeasurementUnit, WeightMeasurementUnit) του κώδικα 11.14.

Για να μετατρέψω τη μονάδα μέτρησης ενός βάρους, στη μονάδα μέτρησης, έστω wUnit, μετατρέπω πρώτα τη μονάδα μέτρησης του βάρους, δηλαδή την weightUnit στην wUnit. Στη συνέχεια, το μέτρο του βάρους που λαμβάνω ως αποτέλεσμα της μετατροπής των μονάδων, το πολλαπλασιάζω με το μέτρο αυτού του βάρους. Σε αυτήν τη λογική βασίζεται η convert(WeightMeasurementUnit) του κώδικα 11.14.

Στη συνάρτηση add ελέγχω αν τα βάρη που πρόκειται να προστεθούν εκφράζονται στην ίδια μονάδα βάρους. Αν όχι μετατρέπω το βάρος που είναι παράμετρος της add στη μονάδα του βάρους που καλεί την add. Εφόσον τα δύο βάρη εκφράζονται στην ίδια μονάδα, η πρόσθεσή τους γίνεται απλώς με την πρόσθεση των μέτρων τους. Το νέο βάρος έχει ως μονάδα την μονάδα του βάρους που κάλεσε την add. Έτσι, η κλήση w1.add(w2) δεν θα επιστρέψει ακριβώς το ίδιο αποτέλεσμα με την κλήση w2.add(w1), καθώς στην πρώτη περίπτωση το βάρος εκφράζεται στη μονάδα της w1 και στη δεύτερη στη μονάδα της w2. Ωστόσο, τα δύο αυτά αθροίσματα παρότι εκφράζονται σε διαφορετικές μονάδες βάρους, αντιπροσωπεύουν ίσα βάρη, πράγμα που πρέπει να επιβεβαιώνεται και από τη μέθοδο equals της κλάσης.

Η μέθοδος equals θα πρέπει να επιστρέφει true για βάρη που είναι ίσα μεταξύ τους, π.χ. το 1 kgr με τα 1000 gr, πρέπει να θεωρούνται ίσα. Έτσι μετατρέπουμε την παράμετρό της στη μονάδα βάρους του αντικειμένου που την καλεί και στη συνέχεια συγκρίνουμε τα μέτρα των δύο βαρών.

Παρόμοια, η hashCode πρέπει να επιστρέφει τον ίδιο κωδικό για δύο βάρη εφόσον η equals τα θεωρεί ίσα. Έτσι μετατρέπουμε κάθε βάρος σε γραμμάρια και παράγουμε κωδικό βασισμένοι στον αριθμό των γραμμαρίων ενός βάρους.

11.9.5 Η κλάση Complex

Κάθε μιγαδικός αριθμός αποτελείται από ένα ζεύγος πραγματικών και συμβολίζεται ως (a,b) ή a+bi, όπου ο πραγματικός a αποτελεί το πραγματικό μέρος του μιγαδικού και ο b το φανταστικό μέρος.
Η πρόσθεση μεταξύ δύο μιγαδικών δίνεται από τον τύπο

$$(x + yi) + (u + vi) = (x + u) + (y + v)i$$

Η αφαίρεση δίνεται από τον τύπο

$$(x + yi) - (u + vi) = (x - u) + (y - v)i$$

Ο πολλαπλασιασμός από τον τύπο

$$(x + yi)(u + vi) = (xu - yv) + (xv - yu)i$$

Ο αντίστροφος ενός μιγαδικού δίνεται από τον τύπο

$$\frac{1}{x + yi} = \frac{x}{x^2 + y^2} - \frac{y}{x^2 + y^2}i$$

Να υλοποιηθεί η κλάση Complex των μιγαδικών αριθμών. Να προστεθούν μέθοδοι για την πρόσθεση, την αφαίρεση, τον πολλαπλασιασμό και τη διαίρεση μιγαδικών. Να υποστηρίζονται και οι δύο συμβολισμοί των μιγαδικών. Να προστεθεί συνάρτηση main στην οποία να διαιρείται ο 2+3i με τον 4+0i και ο 20-4i με τον 3+2i.

Σημείωση: Τα αποτελέσματα των ζητούμενων διαιρέσεων είναι 0.5+0.75i και 4.0-4.0i, αντίστοιχα.

Λύση

```
public class Complex {

    public static int format = 1;
    private double r;
    private double f;

    public Complex(double r, double f) {
        this.r = r;
        this.f = f;
    }

    public Complex(Complex c) {
        this(c.r, c.f);
    }

    public double getR() {
        return r;
    }

    public void setR(double r) {
        this.r = r;
    }

    public double getF() {
        return f;
    }

    public void setF(double f) {
        this.f = f;
    }

    @Override
    public int hashCode() {
        return Double.hashCode(r) + Double.hashCode(f);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
```

```
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    final Complex other = (Complex) obj;
    if (Double.compare(this.r, other.r) != 0) {
        return false;
    }
    return Double.compare(this.f, other.f) == 0;
}

@Override
public String toString() {
    if (r==0 && f==0) return "0";
    String rVal="";
    if (format == 1) {
        if (r!=0)
            rVal+=r;  //8i, 2-81
        if (f>0)
            rVal+=(r!=0?"+":"")+f+"i";
        else
            if (f<0)
                rVal+=f+"i";
    }
    else {
        rVal="("+r+","+f+")";
    }
    return rVal;
}

public Complex add(Complex c) {
    double real=r+c.r;
    double imagin=f+c.f;
    return new Complex(real, imagin);
}

public Complex subtract(Complex c) {
    double real=r-c.r;
    double imagin=f-c.f;
    return new Complex(real, imagin);
}

public Complex multiply(Complex c) {
    double real=r*c.r-f*c.f;
    double imagin=r*c.f+f*c.r;
    return new Complex(real, imagin);
}

public Complex divide(Complex c) {
    if (c.r==0 && c.f==0)
        throw new RuntimeException();
    double real=c.r/(Math.pow(c.r, 2)+Math.pow(c.f, 2));
    double imagin=c.f/(Math.pow(c.r, 2)+Math.pow(c.f, 2));
    Complex divisor= new Complex(real,-imagin);
    return multiply(divisor);
}

public static void main(String[] args) {
    Complex.format=1;
```

```
        Complex c1=new Complex(2,3);
        Complex c2=new Complex(4,0);
        System.out.println(c1.divide(c2));
        c1=new Complex(20,-4);
        c2=new Complex(3,2);
        System.out.println(c1.divide(c2));
        System.out.println(Double.doubleToLongBits(c1.r) ^
Double.doubleToLongBits(c1.r)>>> 32);
    }
}
```

Κώδικας 11.15 *Η κλάση Complex των Μιγαδικών*

Η κλάση Complex στον κώδικα 11.15 διαθέτει τη στατική μεταβλητή format και δύο μεταβλητές στιγμιότυπου, τις r και f. Η format αξιοποιείται από την toString η οποία ανάλογα με την τιμή της format ρυθμίζει ποιος από τους δύο συμβολισμούς των αντικειμένων της Complex θα χρησιμοποιείται, ενώ τα μέλη r και f αναπαριστούν το πραγματικό και φανταστικό μέρος του μιγαδικού. Στη συνέχεια περιλαμβάνει δύο δημιουργούς. Ο πρώτος είναι ο τυπικός δημιουργός που κατασκευάζει μιγαδικούς με είσοδο δύο πραγματικών, ενώ ο δεύτερος είναι δημιουργός αντιγραφής. Οι αναγνώστες, οι ρυθμιστές, η equals και η hashCode της κλάσης είναι τυπικές οπότε δεν τις σχολιάζουμε παραπέρα. Η μέθοδος toString είναι λίγο πιο περίπλοκη, τόσο για να υποστηρίζει τους διαφορετικούς συμβολισμούς των αντικειμένων όσο και για να υποστηρίζει ειδικές περιπτώσεις μιγαδικών όπως οι καθαροί φανταστικοί, π.χ., (0, 5), οι καθαροί πραγματικοί, π.χ. (2, 0) και οι μιγαδικοί με αρνητικό φανταστικό μέρος, π.χ. (2, -8). Οι μέθοδοι add, subtract και multiply αποτελούν καθαρή εφαρμογή των αντίστοιχων τύπων που δόθηκαν στην εκφώνηση. Η μέθοδος divide ελέγχει αρχικά αν ζητείται διαίρεση με το 0. Θεωρούμε εδώ πως η διαίρεση με το 0 δεν ορίζεται. Επομένως, αν μια εφαρμογή επιχειρήσει διαίρεση με το 0, ο κώδικας θα αποκριθεί με την παραγωγή εξαίρεσης, Στη συνέχεια η διαίρεση μεταξύ δύο μιγαδικών x/y γίνεται αφού υπολογίσουμε τον αντίστροφο του διαιρέτη, z=1/y και πολλαπλασιάσουμε το x με το z.

11.9.6 Παιχνίδι ζαριών

Τρεις παίκτες, ο John, ο Jim και ο Jack συναντήθηκαν προκειμένου να παίξουν ένα παιχνίδι ζαριών αποτελούμενο από παρτίδες με τους ακόλουθους κανόνες: Σε κάθε παρτίδα, οι παίκτες ρίχνουν εναλλάξ από 10 ζαριές. Κάθε φορά που ένας παίκτης φέρνει άρτιο αποτέλεσμα αυξάνεται κατά 1 το σκορ του. Στην περίπτωση που δύο ή περισσότεροι παίκτες ισοβαθμήσουν, η παρτίδα επαναλαμβάνεται. Σε κάθε επανάληψη, τα σκορ των παικτών ενημερώνονται αθροιστικά. Όταν στο τέλος μιας παρτίδας, όλοι οι παίκτες έχουν διαφορετικό σκορ, το παιχνίδι λήγει και νικητής ανακηρύσσεται ο παίκτης με το μεγαλύτερο σκορ.

Οι παίκτες όμως δεν είναι το ίδιο τυχεροί. Ο John φέρνει κάθε δυνατό αποτέλεσμα με πιθανότητα 1/6, ο Jim δεν φέρνει ποτέ 5 και στη θέση του φέρνει 6 με πιθανότητα 1/3, ο Jack δεν φέρνει ποτέ 6 και στη θέση του φέρνει 5 με πιθανότητα 1/3. Όλα τα άλλα αποτελέσματα, τόσο ο Jim όσο και ο Jack τα φέρνουν με πιθανότητα 1/6.

Να υλοποιηθεί η κλάση DiceGame που διαθέτει τη μέθοδο public DicePlayer play() που προσομοιώνει ένα παιχνίδι ζαριών και επιστρέφει τον νικητή. Να προστεθεί main που καλεί την play() και εμφανίζει τον νικητή και το σκορ του.

Υπόδειξη: Δημιουργήστε μια κλάση που αναπαριστά έναν παίκτη ζαριού. Δημιουργήστε τρία αντικείμενα της κλάσης, από ένα για τον John, τον Jim και τον Jack.

Λύση

Για την υλοποίηση αυτής της άσκησης, χρησιμοποιούμε συνολικά τρεις κλάσεις, την Lib, την DicePlayer και την DiceGame. Η Lib περιλαμβάνει στατικές συναρτήσεις και σταθερές γενικού χαρακτήρα, η DicePlayer μοντελοποιεί τον παίκτη ζαριού και η DiceGame το παιχνίδι ζαριών έτσι όπως περιγράφηκε στην εκφώνηση.

```
import java.util.Random;

public class Lib {
    public final static double EPSILON=0.000001;
    public final static Random gen=new Random();
```

```
    public static boolean approximateEquals(double f1, double f2) {
        return Math.abs(f2 - f1) < EPSILON;
    }

    public static double sum(double[] t) {
        double sum=0;
        for (double d:t)
            sum+=d;
        return sum;
    }
}
```

Κώδικας 11.16 *Η κλάση Lib*

Όπως φαίνεται στον κώδικα 11.16, στην Lib ορίζονται στατικές μεταβλητές και συναρτήσεις γενικού σκοπού. Πιο συγκεκριμένα, ορίζεται μια συνάρτηση προσεγγιστικής ισότητας, μια συνάρτηση που υπολογίζει το άθροισμα των στοιχείων μονοδιάστατου πίνακα, μια γεννήτρια τυχαίων αριθμών και η σταθερά EPSILON που αξιοποιείται από την προσεγγιστική ισότητα. Όπως θα δούμε, όλα τα μέλη της Lib χρησιμοποιούνται από την κλάση DicePlayer. Με αυτήν την έννοια θα μπορούσαν να είναι μέλη της DicePlayer. Τοποθετήθηκαν όμως σε χωριστή κλάση λόγω του γενικού χαρακτήρα τους. Η προσεγγιστική ισότητα για παράδειγμα μπορεί να αξιοποιηθεί και από άλλα πρότζεκτ που δεν έχουν καμία σχέση με το παιχνίδι ζαριών. Ο σχεδιασμός που προτείνουμε μας διευκολύνει ώστε σε αυτά, τα προτζεκτ να εισάγουμε μόνο την Lib και όχι την DicePlayer.

```
import java.util.Arrays;
import java.util.Objects;

public class DicePlayer {

    private final double[] distribution;
    private final String name;

    public DicePlayer(String name, double[] distribution) {

        if (distribution.length != 6
                || !Lib.approximateEquals(Lib.sum(distribution), 1)) {
            throw new RuntimeException();
        }
        this.name = name;
        this.distribution = distribution;
    }

    public DicePlayer(String name) {
        this(name, new double[]{1d / 6, 1d / 6, 1d / 6, 1d / 6, 1d / 6, 1d
/ 6,});
    }

    public int roll() {
        double value = Lib.gen.nextDouble();
        double sum = 0;

        for (int i = 0; i < distribution.length; i++) {
            sum += distribution[i];
            if (value <= sum) {
                return i + 1;
            }
        }
        return 0;
    }

    public String getName() {
```

```
            return name;
        }

        @Override
        public int hashCode() {
            return Arrays.hashCode(this.distribution)+this.name.hashCode();
        }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) {
                return true;
            }
            if (obj == null) {
                return false;
            }
            if (getClass() != obj.getClass()) {
                return false;
            }
            final DicePlayer other = (DicePlayer) obj;
            if (!Objects.equals(this.name, other.name)) {
                return false;
            }
            return Arrays.equals(this.distribution, other.distribution);
        }
    }
```

Κώδικας 11.17 *Η κλάση DicePlayer*

Ο δημιουργός της κλάσης DicePlayer του κώδικα 11.17 παρέχει τη δυνατότητα να κατασκευάζονται παίκτες με διαφορετικά χαρακτηριστικά ζαριάς. Λαμβάνει 2 παραμέτρους, μια για το όνομα του παίκτη και μια για την κατανομή που καθορίζει τα αποτελέσματα των ρίψεων ζαριού για τον συγκεκριμένο παίκτη. Ο δημιουργός ελέγχει ώστε το σύνολο των πιθανοτήτων για να φέρει ένα αποτέλεσμα από 1 έως 6 ο παίκτης να είναι ίσο με 1. Επίσης, παρέχεται ένας δημιουργός με παράμετρο μόνο το όνομα του παίκτη που κατασκευάζει έναν παίκτη με ίση πιθανότητα για κάθε πιθανή ζαριά.

Προσέξτε πως τα μέλη name και distribution είναι final ενώ δεν παρέχονται για αυτά συναρτήσεις ρυθμιστές. Έτσι μετά τη δημιουργία ενός παίκτη δεν υπάρχει η δυνατότητα να αλλάξει το όνομά του ή τα χαρακτηριστικά της ζαριάς του.

Η συνάρτηση roll βασίζεται στον αλγόριθμο που περιγράφεται στον κώδικα 8.4.

Οι equals και hashCode είναι σχετικά απλές, κρίσιμες όμως στην συγκεκριμένη εφαρμογή, καθώς όπως θα δούμε στην DiceGame χρησιμοποιείται ένα HashMap που τις αξιοποιεί.

```
import java.util.HashMap;

public class DiceGame {

    private final int noOfPlayers;
    private final int noOfRoll;

    private final DicePlayer john = new DicePlayer("John");
    private final DicePlayer jim = new DicePlayer("Jim", new double[]{1d /
6, 1d / 6, 1d / 6, 1d / 6, 0, 2d / 6});
    private final DicePlayer jack = new DicePlayer("Jack", new double[]{1d /
6, 1d / 6, 1d / 6, 1d / 6, 2d / 6, 0});

    private final HashMap<DicePlayer, Integer> scores = new HashMap<>();

    public DiceGame(int noOfPlayers, int noOfRoll) {
        this.noOfPlayers = noOfPlayers;
        this.noOfRoll = noOfRoll;
```